Credit Card Approval Prediction

Project Description:

The Credit Card Approval Prediction system is a Machine Learning-based web application designed to automate the credit card approval process by predicting whether an applicant is eligible for a credit card based on personal, financial, and employment information.

The application allows users to enter applicant information such as age, gender, annual income, employment status, family status, education level, housing type, and other financial details through a user-friendly web interface.

The project is developed using Python, Flask, Scikit-learn, Pandas, and NumPy. Data visualization is performed using Matplotlib and Seaborn, while the trained model is integrated into a Flask web application to provide instant predictions

The Credit Card Approval Prediction system reduces manual verification efforts, improves decision-making speed, and minimizes errors by providing automated, data-driven credit approval recommendations.

Scenarios

Scenario 1: New Credit Card Application

A customer visits the Credit Card Approval Prediction web application and enters personal details, employment information, annual income, and credit-related details. After submitting the application, the system validates the information, processes the data using the trained Machine Learning model, and predicts whether the applicant is eligible for a credit card.

Scenario 2: Bank Officer Verification

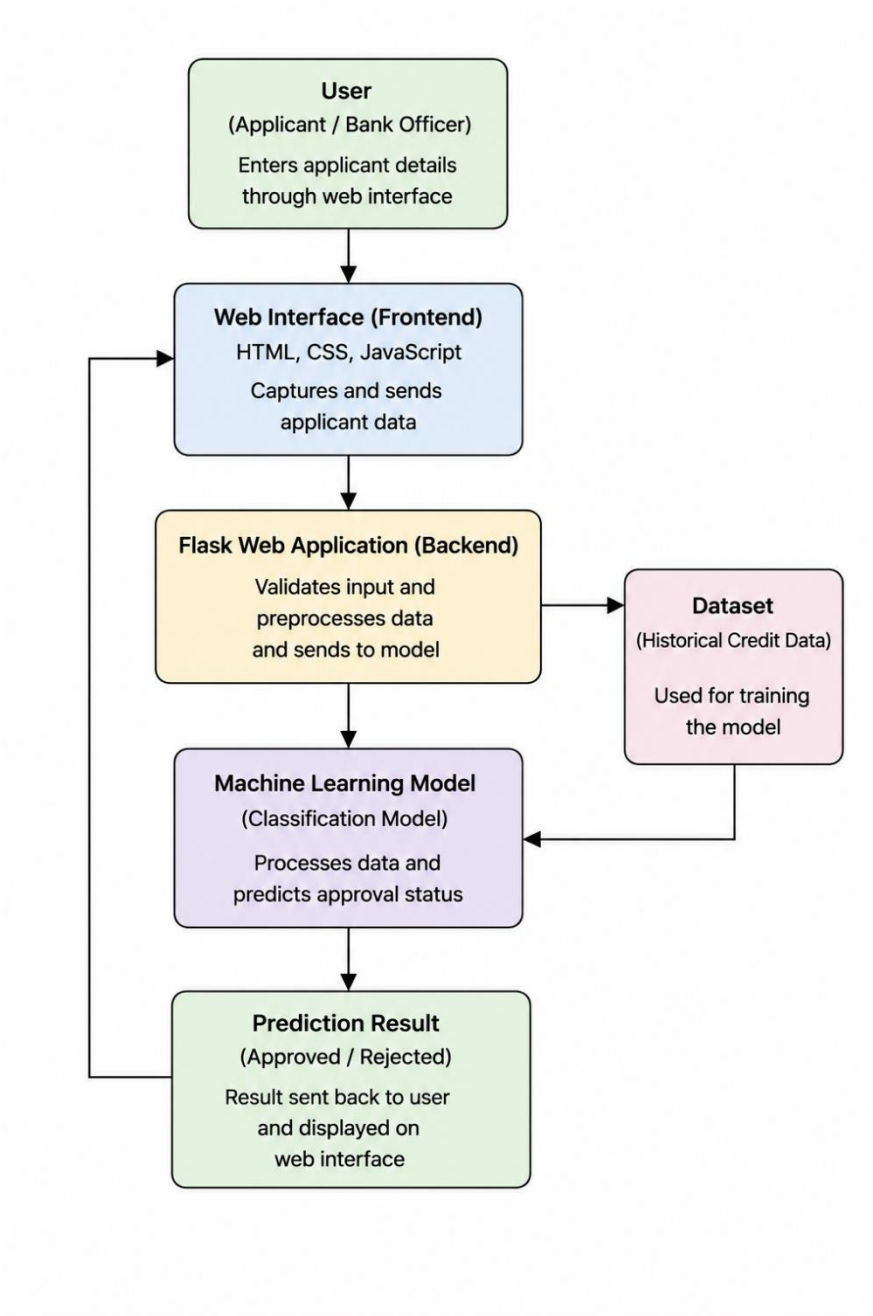
A bank officer receives a new credit card application and uses the prediction system to evaluate the applicant's eligibility. The model analyzes the

submitted information and instantly predicts the approval status, helping the officer make faster and more accurate decisions.

Scenario 3: Bulk Applicant Screening

A financial institution processes multiple credit card applications using the prediction system. The Machine Learning model evaluates each applicant based on historical data patterns, enabling the organization to automate the screening process, reduce manual effort, and improve operational efficiency.

Technical Architecture



The Credit Card Approval Prediction system follows a modular three-tier architecture consisting of the Presentation Layer, Application Layer, and Machine Learning Layer.

The Presentation Layer provides a responsive web interface developed using HTML, CSS, and Flask templates, allowing users to enter applicant details and view prediction results.

The Application Layer is implemented using the Flask framework, which handles user requests, validates input data, performs preprocessing.

The Machine Learning Layer consists of data preprocessing techniques, feature engineering, model training, evaluation, and prediction.

Pre-requisites

Before developing the Credit Card Approval Prediction project, the following knowledge and tools are recommended:

- Python Programming Fundamentals
- Machine Learning Concepts and Classification Algorithms
- Flask Framework for Web Development
- Pandas and NumPy for Data Processing
- Matplotlib and Seaborn for Data Visualization
- Scikit-learn for Machine Learning Model Development
- Git and GitHub for Version Control
- Visual Studio Code or PyCharm IDE
- Basic HTML and CSS Knowledge
- IBM Watson Machine Learning / Flask Deployment (if applicable)

Project Workflow

Milestone 1: Dataset Collection and Environment Setup

- **Activity 1.1:** Collect the Credit Card Approval dataset from a reliable source.
- **Activity 1.2:** Analyze the dataset and understand the available features and target variable.
- **Activity 1.3:** Design the system architecture showing the interaction between users, Flask application, Machine Learning model, and dataset.
- **Activity 1.4:** Set up the development environment by installing Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, and other required libraries.

Milestone 2: Data Analysis and Preprocessing

- **Activity 2.1:** Perform Exploratory Data Analysis (EDA) to understand the dataset and identify important features.
- **Activity 2.2:** Clean the dataset by handling missing values, removing duplicates, encoding categorical variables, and preparing the data for model training.

Milestone 3: Model Development

- **Activity 3.1:** Train multiple Machine Learning classification algorithms.
- **Activity 3.2:** Compare model performance using evaluation metrics and select the best-performing model.
- **Activity 3.3:** Save the trained model for deployment.

Milestone 4: Flask Application Development

- **Activity 4.1:** Develop the frontend using HTML, CSS, and Flask templates.
- **Activity 4.2:** Implement the Flask backend to load the trained model and process prediction requests.
- **Activity 4.3:** Display the prediction result to the user through the web interface.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the application using sample applicant data and verify prediction accuracy.
- **Activity 5.2:** Deploy the application locally or on a cloud platform and ensure all project components function correctly.

Milestone 6: Conclusion

Summarize the project outcomes, evaluate the prediction model's performance, and discuss future improvements such as explainable AI, cloud deployment, and enhanced prediction accuracy.

Milestone 1: Environment Setup and Data Pipeline

In this milestone, we focus on establishing an isolated development workspace, downloading the necessary credit risk datasets, and building the initial merging pipeline that programmatically engineers binary default targets from historical repayment accounts.

Activity 1.1: Set Up the Development Environment

To isolate project dependencies and prevent conflict with system-level packages, create and activate a Python virtual environment (venv).

- **Step 1 — Create a Virtual Environment**: Open a terminal in the root directory and run

```powershell
python -m venv venv
```

- **Step 2 — Activate the Environment**:
  - **PowerShell**: .\venv\Scripts\Activate.ps1
  - **Command Prompt**: venv\Scripts\activate.bat
- **Step 3 — Install Dependencies**: Install essential libraries for data engineering, model tuning, and web serving

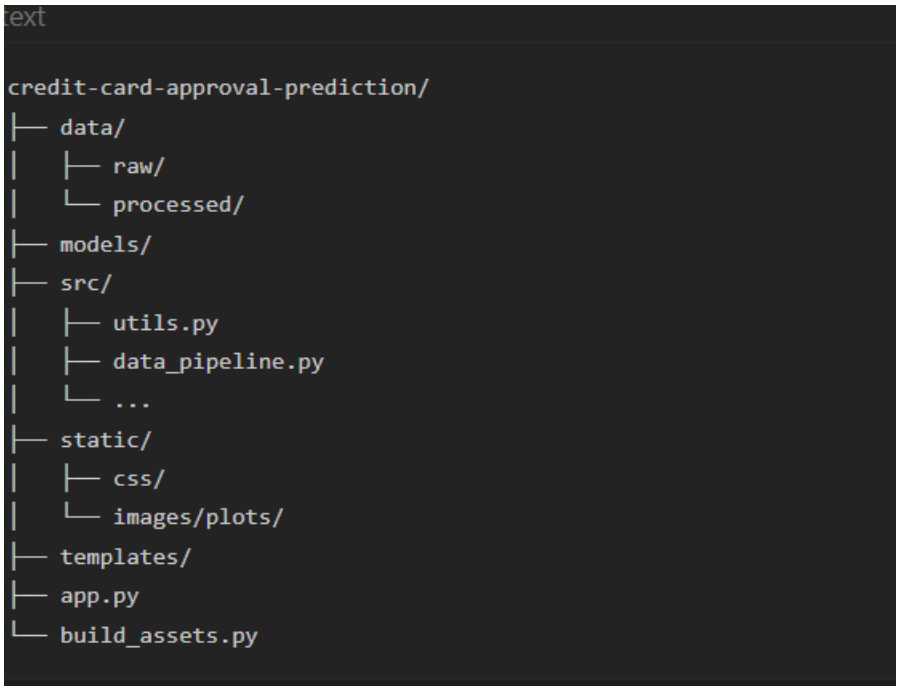
```powershell
pip install -r requirements.txt
```

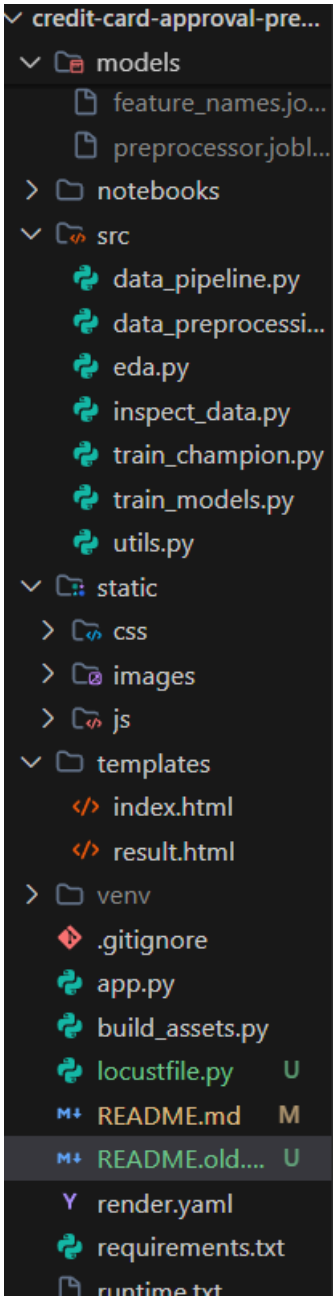
NOTE

Key Dependencies Installed:

- pandas & numpy (Data manipulation)
- scikit-learn & xgboost (Machine learning & preprocessing)
- matplotlib & seaborn (Visualization plots)
- flask & gunicorn (Web application server)
- joblib (Model serialization)

- **Step 4 — Setup Project Directory Structure**: Ensure the folders match the workspace layout, separating raw data, processed splits, static web assets, and training templates:





Activity 1.2: Download Raw Datasets

Configure dataset download utilities in src/utils.py to fetch application demographic files and credit records programmatically, bypassing manual downloads:

```
# target URLs for the Kaggle Credit Card Approval Dataset
URL_APPLICATION_RECORD = "https://raw.githubusercontent.com/shiraz-
30/Credit-Card-Approval-Prediction-Model/master/application_record.csv"
URL_CREDIT_RECORD = "https://raw.githubusercontent.com/shiraz-30/Credit-
Card-Approval-Prediction-Model/master/credit_record.csv"
```

Activity 1.3: Clean and Merge Data (Repayment Target Engineering)

In credit risk modeling, we must define who is a "good" borrower and who is "bad". The dataset contains monthly status logs for each account:

- C: Paid off that month.
- X: No loan activity that month.
- 0: 1–29 days overdue.
- 1: 30–59 days overdue.
- 2: 60–89 days overdue.
- 3: 90–119 days overdue.
- 4: 120–149 days overdue.
- 5: 150+ days overdue.

We engineer a binary classification target in src/data_pipeline.py:

- **TARGET = 1 (High Risk / Reject)**: If the client ever went 30+ days overdue (1, 2, 3, 4, or 5).
- **TARGET = 0 (Low Risk / Approve)**: If the client maintained standard repayment (C, X, 0).

We then remove duplicates from the demographic table based on ID and run an inner join to merge demographic data with the engineered targets.

```
# Target Engineering logic snippet from src/data_pipeline.py
bad_statuses = {'1', '2', '3', '4', '5'}
credit_df['IS_BAD'] = credit_df['STATUS'].apply(lambda x: 1 if x in
bad_statuses else 0)

# Group by ID and check if they ever had a bad month
grouped = credit_df.groupby('ID')['IS_BAD'].max().reset_index()
grouped.rename(columns={'IS_BAD': 'TARGET'}, inplace=True)

# Merge datasets
merged_df = pd.merge(app_df, grouped, on="ID", how="inner")
```

Milestone 2: Preprocessing and Pipeline Serialization

In this milestone, we engineer clean feature inputs from raw database records, define a column transformer to automate pipeline steps, split datasets to prevent leakage, and serialize the fitted preprocessor for live endpoint inference.

Activity 2.1: Feature Engineering and Selection

Raw features require conversion before mathematical operations.

- **Age**: Convert the negative days field DAYS_BIRTH to positive age in years: $$\text{AGE} = -\frac{\text{DAYS_BIRTH}}{365.25}$$
- **Years Employed**: Extract positive employment years from DAYS_EMPLOYED. Set positive values (which represent anomalous codes for unemployment in the dataset) to 0 and flag them with an IS_UNEMPLOYED indicator.
- **Feature Dropping**: Drop variables with zero variance or identifier leakage:
    - ID (prevents model overfitting to applicant identifiers).
    - FLAG_MOBIL (100% of applicants have mobiles; zero information variance).

Activity 2.2: Build Scikit-learn ColumnTransformer Pipeline

Create a robust, structured data scaling and encoding pipeline in src/data_preprocessing.py to process columns by data type:

```
# Imputation & encoding configurations in src/data_preprocessing.py
numerical_features = ['AMT_INCOME_TOTAL', 'AGE', 'YEARS_EMPLOYED', 'CNT_CHILDREN', 'CNT_FAM_MEMBERS']
categorical_features = ['CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE',
'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE', 'OCCUPATION_TYPE']

# Pipelines
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_pipeline, numerical_features),
        ('cat', cat_pipeline, categorical_features)
    ],
    remainder='passthrough'  # Keeps pre-formatted binary indicators untouched
)
```

Activity 2.3: Data Splitting and Preprocessor Serialization

- **Stratification**: Divide the data into **80% training** and **20% testing** splits. Because default cases represent a small minority of the population (~12%), stratified splitting is required to keep identical label distributions in both slices.
- **Fitting and Saving**: Fit the pipeline exclusively on the training partition to prevent test data leakage. Save the compiled outputs:

```
# Fit preprocessor on train, transform both splits
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Serialize the fitted pipeline and feature list
joblib.dump(preprocessor, "models/preprocessor.joblib")
joblib.dump(all_feature_names, "models/feature_names.joblib")
```

Milestone 3: Model Training, Tuning, and Selection

In this milestone, we run hyperparameter search grids across four standard classification models, evaluate their predictions, choose the ideal model for production, and save diagnostic plots.

Activity 3.1: Model Training and Cross-Validation

In src/train_models.py, we train **Logistic Regression**, **Decision Trees**, **Random Forests**, and **XGBoost**. We use GridSearchCV with **3-Fold Stratified Cross-Validation** to optimize hyperparameters for credit score default sensitivity (ROC-AUC).

To handle the 88% approved / 12% default imbalance, configure class weight balancing:

- Logistic Regression & Random Forest:
  Set class_weight='balanced'.
- XGBoost: Set scale_pos_weight=7.5.

Activity 3.2: Performance Metric Evaluation and Champion Selection

Compare the algorithms on the held-out test split:

Model Classifier	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	57.89%	13.66%	48.48%	0.2132	0.5501
Decision Tree	67.40%	21.22%	65.27%	0.3203	0.7181
XGBoost	78.77%	29.56%	58.16%	0.3920	0.7510
Random Forest (Champion)	**80.44%**	**30.99%**	**53.96%**	**0.3937**	**0.7559**

- **Champion Selection**: The **Random Forest** is selected as the production model due to it having the highest F1-Score (0.3937), highest Accuracy (80.44%), and best ROC-AUC (0.7559).
- **Save Model Weights**: Serialize the best classifier to models/best_model.joblib.

Activity 3.3: EDA & Model Comparison Plots Generation

The script generates the visual charts displayed in the dashboard:

1. 01_target_distribution.png (Class balance percentages).

2. 02_income_distribution.png (Income density split by credit risk).
3. 05_rejection_by_education.png (Default rates across degrees).
4. 06_correlation_heatmap.png (Linear associations).
5. 07_model_comparison.png (Accuracy and Recall comparisons).
6. 08_best_model_confusion_matrix.png (True positive vs. false positive splits).
7. 09_feature_importances.png (Which demographics impact score decisions)

Milestone 4: Flask Backend & API Development

In this milestone, we write the Flask web server in app.py to manage requests, run real-time inference, and output prediction scores.

Activity 4.1: Load Assets and Define Portals

Load the serialized preprocessor and model at startup:

```
# Startup asset loading in app.py
preprocessor = joblib.load("models/preprocessor.joblib")
model = joblib.load("models/best_model.joblib")
```

Activity 4.2: Define Route Handlers

- **Home Endpoint (/)**: Renders the portal, parses the metrics comparison CSV, and builds the performance table.
- **Prediction Endpoint (/predict)**: Receives the applicant form details, formats them into a DataFrame matching training features, applies the pre-fitted preprocessing transformations, runs the Random Forest classifier, and calculates risk scores.

```
# 2. Transform the raw input data using our saved preprocessor pipeline
processed_input = preprocessor.transform(input_data)

# 3. Predict class (0 = Approved, 1 = Rejected)
prediction = int(model.predict(processed_input)[0])

# 4. Extract probability of default risk
prob_default = model.predict_proba(processed_input)[0][1]
approval_score = round((1 - prob_default) * 100, 1)
```

Milestone 5: Classic Corporate Frontend Dashboard

In this milestone, we design a modern, professional dark/slate-themed dashboard layout for model monitoring and applicant submissions.

Activity 5.1: Create Sidebar Dashboard Layout

The frontend interface templates/index.html uses a split layout:

- **Left Sidebar**: Renders corporate branding (🏢 CREDIT SCORE INC.) and navigation buttons to toggle central views.
- **Central Panels**: Uses vanilla JavaScript in static/js/app.js to toggle display classes dynamically between:
    1. **Applicant Demographics Form**: Full form input options.
    2. **Model Performance**: Displays the cross-validation comparison table and visual metric charts.
    3. **Dataset Insights**: Embeds demographic distributions and heatmap correlation plots.

Activity 5.2: Create Score Assessment Page

The application decision is displayed on templates/result.html:

- **Status Banners**: Renders green for □ APPLICATION APPROVED or red for ✗ APPLICATION REJECTED.
- **Animated Circular Gauge**: An SVG circle that animates on page load. Its progress corresponds to the calculated approval probability.

- **Risk Categorization**: Displays a badge representing the risk category:
    - **Low Risk**: default probability $\le$ 25%
    - **Medium Risk**: default probability between 25% and 60%
    - **High Risk**: default probability $>$ 60%
- **Decision Rationale**: Text summarizing key factors that influenced the machine learning model's output.

Milestone 6: Deployment & Verification

In this milestone, we run build pipelines to verify the backend code, execute local testing, and deploy to a cloud server.

Activity 6.1: Run Production Build Pipeline

Compile all assets, fetch raw tables, process targets, train the Random Forest champion model, and save files using build_assets.py:

```
2026-07-02 22:15:30,240 - INFO - Step 4/4: Training production
model...
2026-07-02 22:15:31,892 - INFO - Fitting champion Random Forest on
training data...
2026-07-02 22:15:32,940 - INFO - Production model successfully saved
to models/best_model.joblib
2026-07-02 22:15:32,955 - INFO -
================================================
2026-07-02 22:15:32,955 - INFO - ASSETS BUILD COMPLETED SUCCESSFULLY
2026-07-02 22:15:32,955 - INFO -
================================================
```

Activity 6.2: Local Development Server Run

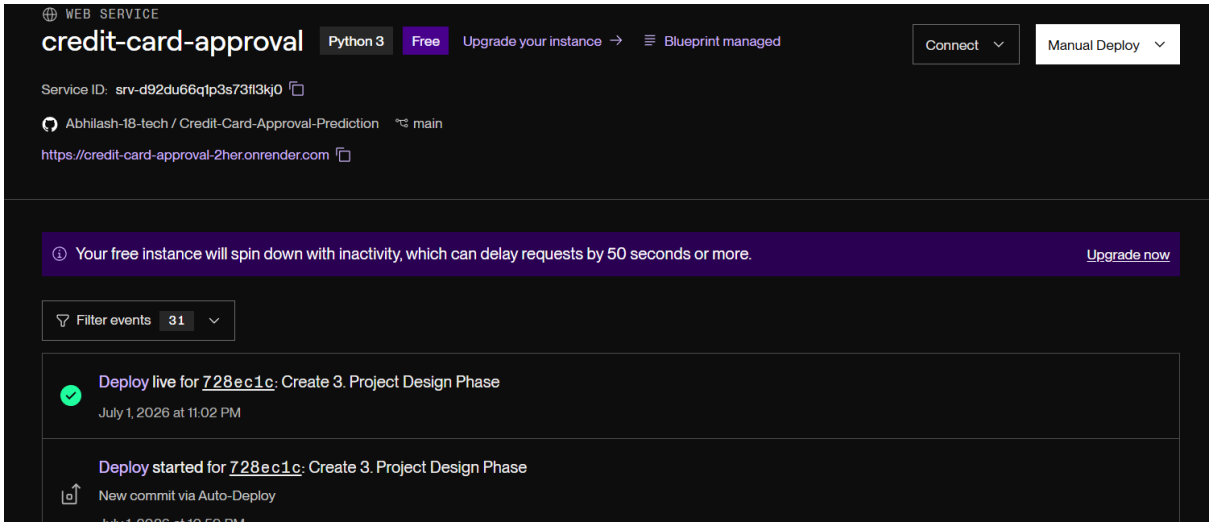
Start the local server and navigate to http://127.0.0.1:5000

```
python app.py
```

Activity 6.3: Production Cloud Deployment (Render)

Deploy the application publicly to **Render** using render.yaml:

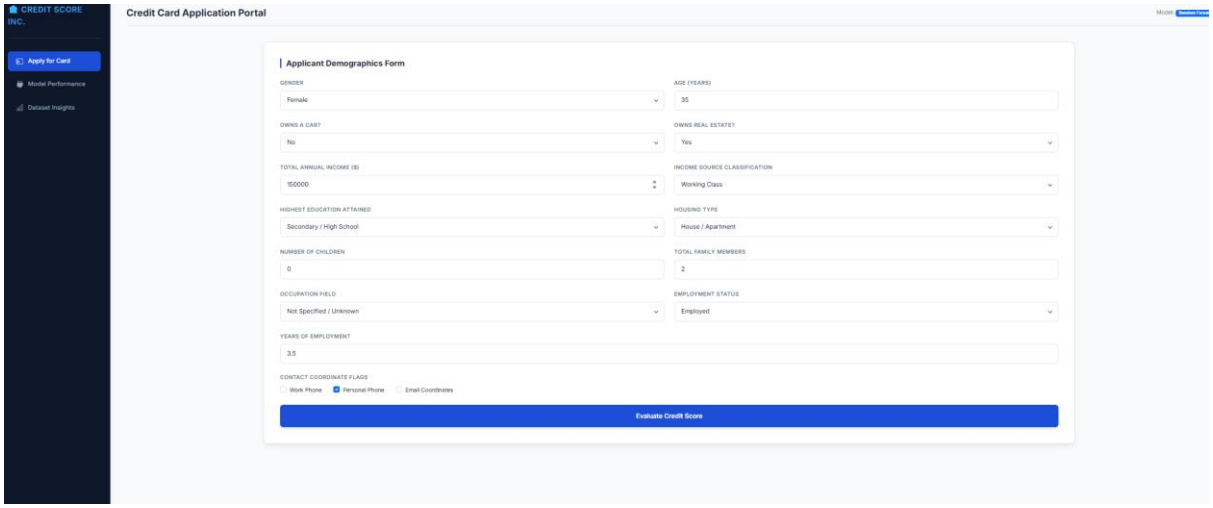
```
services:
  - type: web
    name: credit-card-approval-prediction
    env: python
    buildCommand: "pip install -r requirements.txt && python
build_assets.py"
    startCommand: "gunicorn app:app"
```



Exploring the Website's Web Pages

Home / Dashboard Page (Form View)

Description: The landing page displays a vertical navigation menu and the demographics form. Input fields include annual income, employment years, and education level. Form fields default to a low-risk profile for testing.

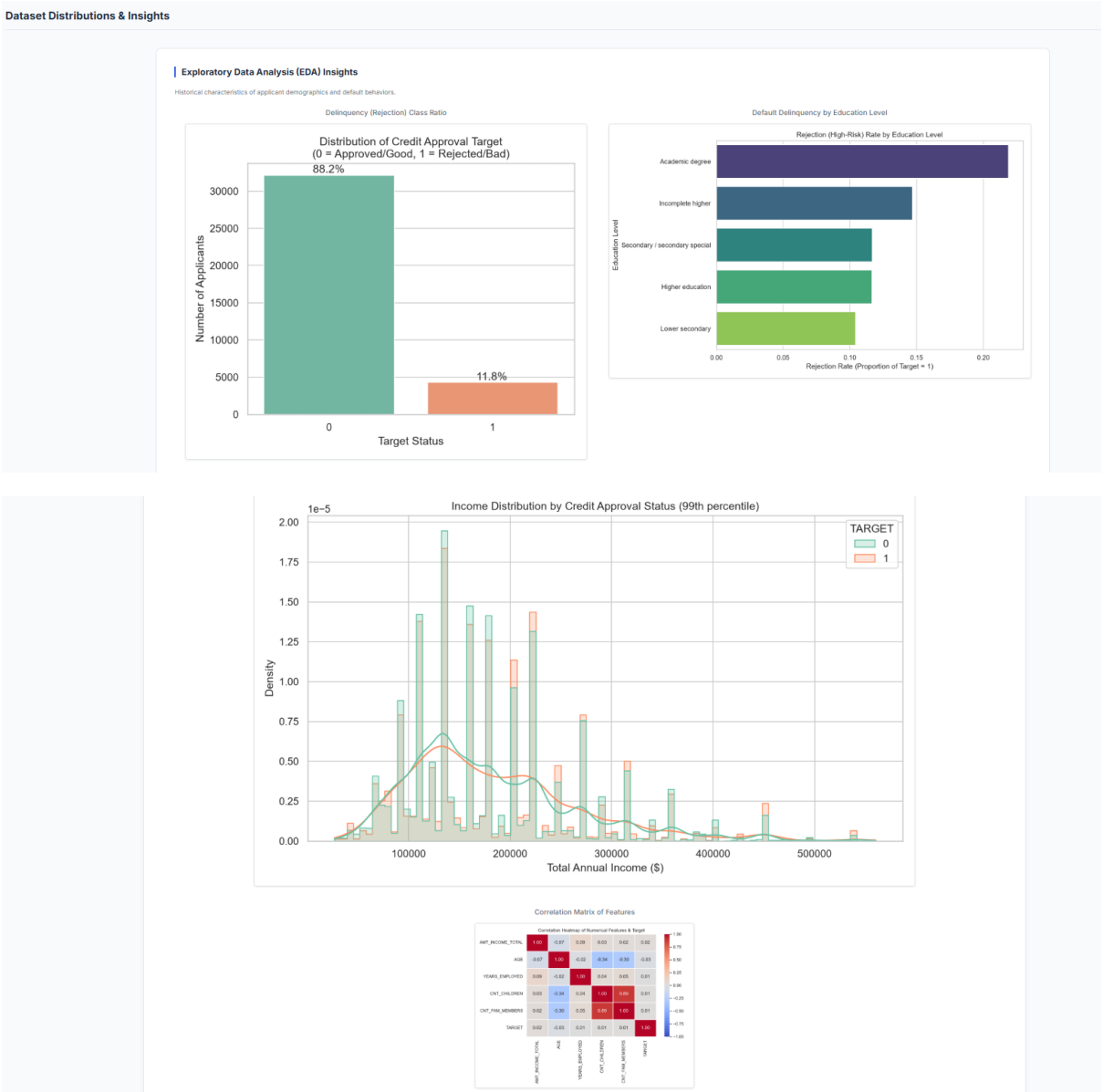


Model Performance Panel

Description: Selecting **Model Performance** toggles the central view to display the model comparison table. The champion Random Forest row is highlighted. Below the table, the interface embeds the metric comparison bar chart, the confusion matrix, and feature importances.

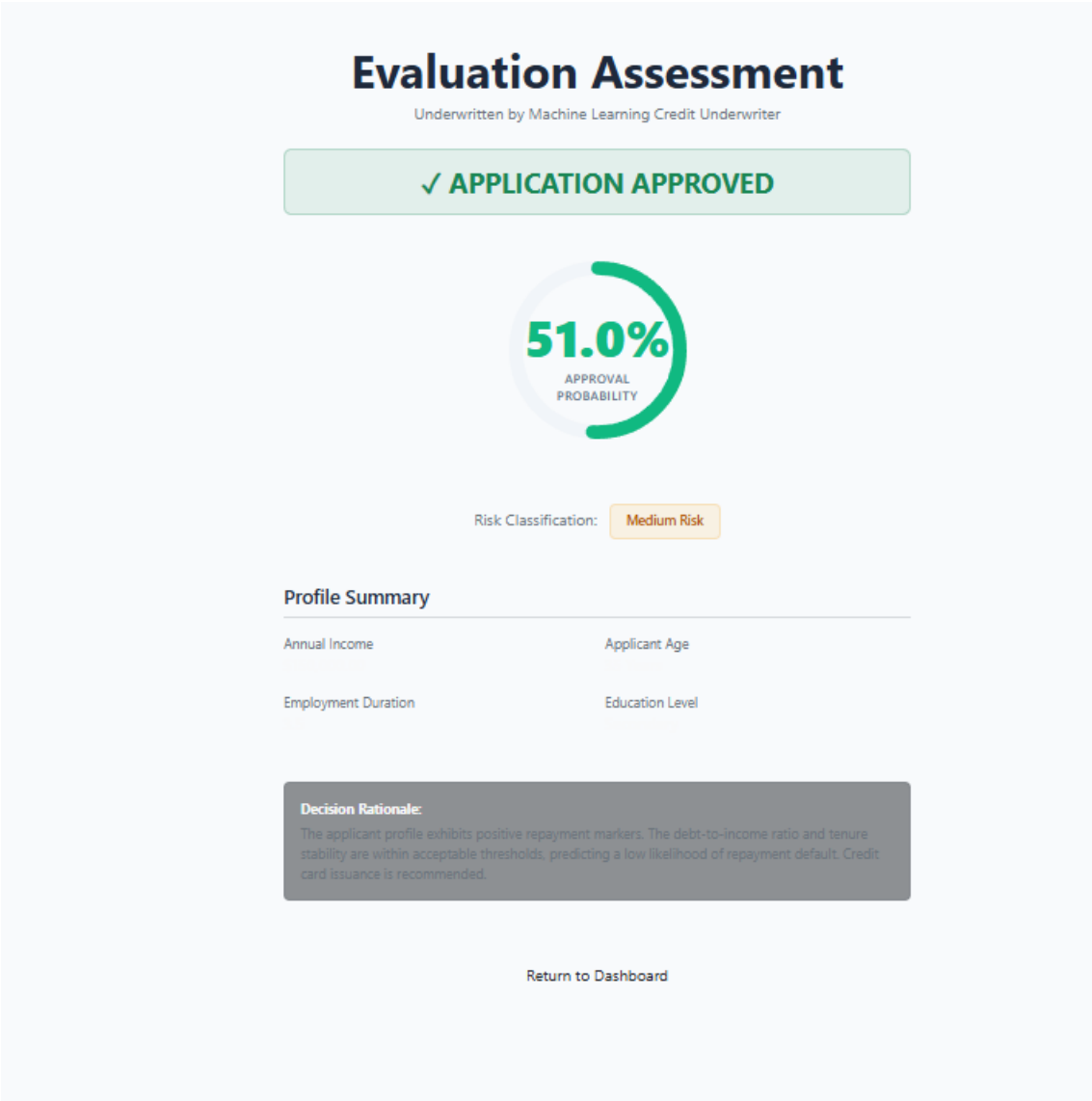
Dataset Insights Panel

Description: Selecting Dataset Insights displays demographic distribution graphs from our Exploratory Data Analysis (EDA). This includes default ratios, income distributions, and a correlation heatmap.



Results Evaluation Page (Approved Case)

Description: Submitting a low-risk profile (e.g., higher education, stable employment) triggers the approved assessment view. It renders a green APPLICATION APPROVED banner, a high score percentage on the circular gauge, a Low Risk badge, and an explanation of the approval decision.



Conclusion

The Credit Card Approval Prediction System provides a complete machine learning solution for banking credit risk assessments. By processing repayment histories into binary labels, we trained a Random Forest model that achieves an accuracy of 80.44% while managing class imbalance using balanced weights.

The Flask application and classic corporate dashboard provide a clean interface for underwriting staff. In the future, this system could be expanded to support database integration (e.g., PostgreSQL for audit logging), real-time applicant data ingestion via credit agency APIs, and automated bias detection checks.